



BugBoard26

Piattaforma di gestione collaborativa
per lo sviluppo software

Simone Parente Martone
Mario Penna
Michela Pollio

OBIETTIVI E REQUISITI

Gestione delle issue senza passaggi superflui

01 SEMPLICE

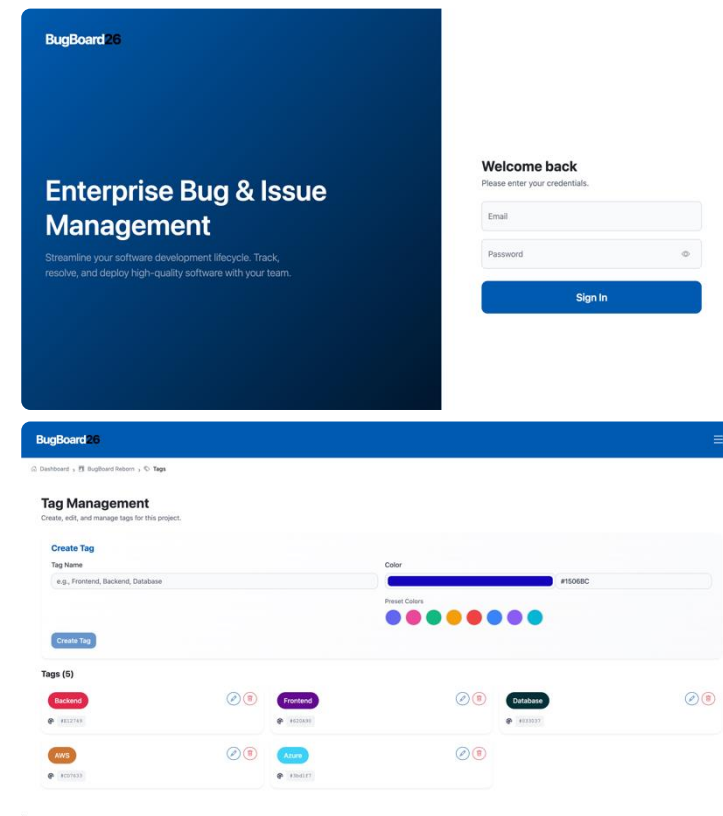
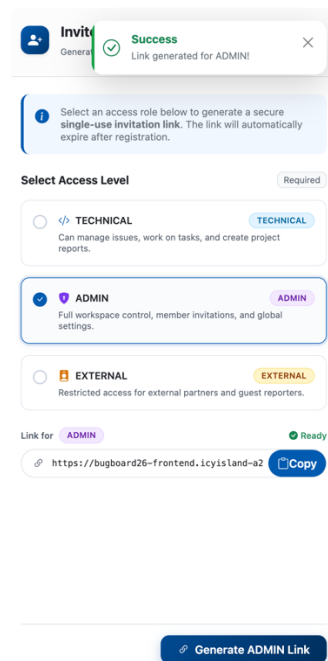
Flussi essenziali e operazioni principali raggiungibili senza formazione specifica.

02 SICURO

BCrypt, sessioni stateless JWT e protezioni applicative contro XSS e SQL injection.

03 EFFICIENTE

Filtri dinamici, report automatici e allegati gestiti tramite cloud storage.



Cinque criteri guidano le scelte tecnologiche

1

Usabilità

Intuitività
Responsiveness
Feedback chiari

2

Efficienza

Prestazioni elevate
Ricerca reattiva
Paginazione

3

Affidabilità

Container
Cloud-Native
Alta Disponibilità

4

Sicurezza

Autenticazione
Autorizzazione
Role-based access

5

Manutenibilità

Test automatici
GitHub Actions
SonarQube

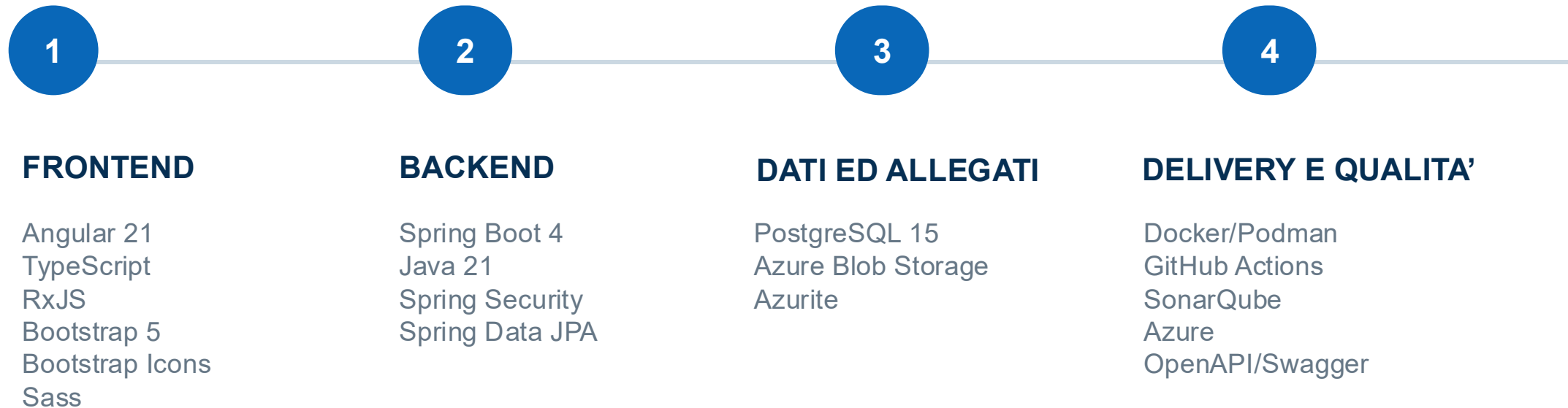
Tre profili, privilegi distinti

RUOLO	CONTESTO	FREQUENZA D'USO	PRIVILEGI	OBIETTIVO
Tecnici	Sviluppatori / tester	Uso quotidiano	Lettura e scrittura su task e etichette	Risoluzione e assegnazione
Amministratori	Team leader	Uso regolare	Accesso completo e inviti	Coordinamento e supervisione
Esterni	Stakeholder / clienti	Uso saltuario	Sola lettura	Monitoraggio e conformità

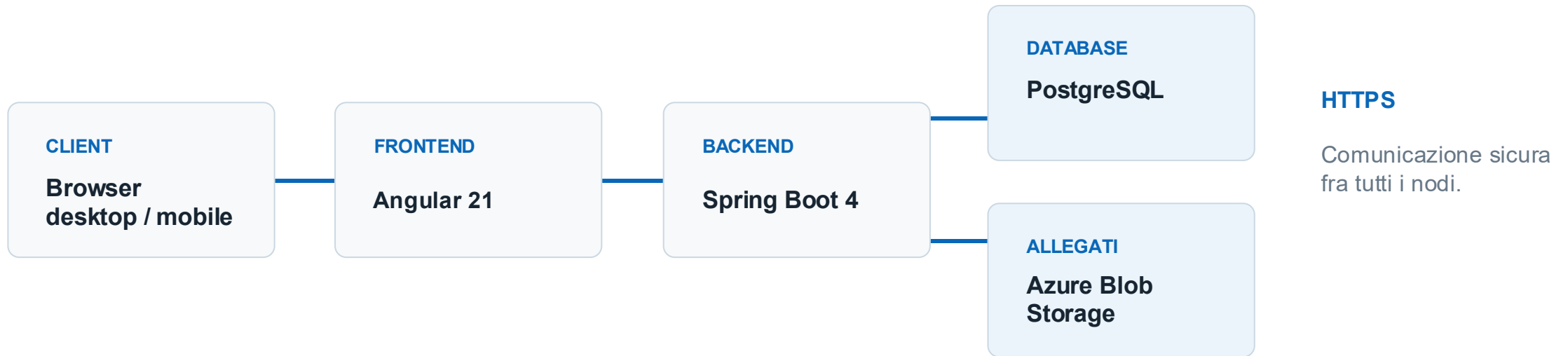
Il modello dei permessi limita le operazioni al perimetro reale di ciascun attore.

ARCHITETTURA

End-to-End: Dallo sviluppo locale al deploy in Cloud



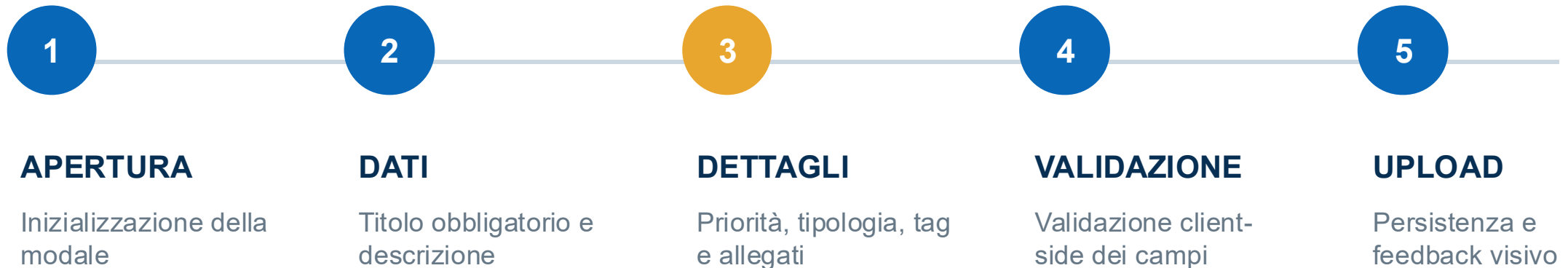
Architettura Multi-Tier per garantire il disaccoppiamento



La separazione delle responsabilità facilita manutenzione, scalabilità e sostituzione dei singoli componenti.

DESIGN E SVILUPPO

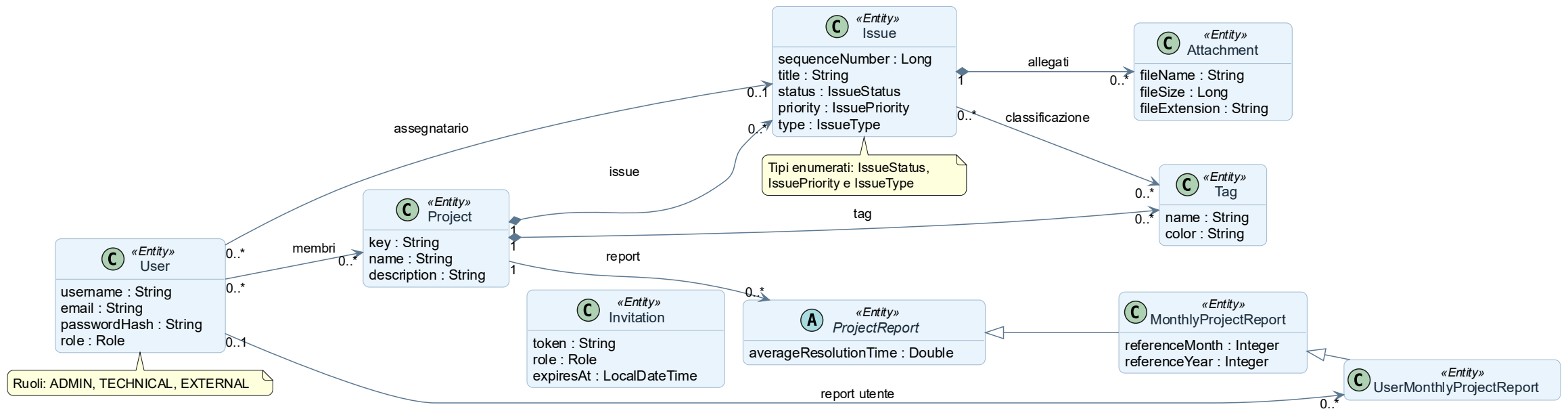
Creare una issue: dal form alla persistenza



VINCOLI RILEVANTI

Titolo e descrizione obbligatori · priorità e tipologia predefinite · allegati con limite dimensionale · validazione client-side

Il modello di dominio supporta i principali casi d'uso



Schema di persistenza dei dati



Registrazione di un nuovo utente



NOTE

È presente un meccanismo di cleanup degli inviti scaduti, configurabile tramite la variabile *fixedDelay* in `InvitationService`

Classificazione e ordinamento avvengono nella stessa vista

BugBoard26

Dashboard > BugBoard Reborn

BugBoard Reborn

Issues Tracker

Manage TagsView ReportNew Issue

Search issues by keyword...

All Types

All Priorities

All Statuses

Export

#	Title ↓	Type	Priority	Status	Assignee
3	Aggiornamento della grafica dell'Header	★ Feature	Medium	TO DO	Unassigned
4	Aggiunta esportazione PDF delle segnalazioni	★ Feature	Low	TO DO	Unassigned
1	Fix del bug di autenticazione JWT	🐛 Bug	Highest	IN PROGRESS	mario.rossi
2	Ottimizzazione query SQL per i report	★ Feature	High	IN PROGRESS	mario.rossi
5	Refactoring dei fogli di stile CSS	📄 Documentation	Medium	TO DO	Unassigned

< 1 >

ASSOCIAZIONE N:M

Selezione multipla dei filtri

Selezione multipla con aggiornamento immediato

ORDINAMENTO

Click-to-sort invertibile

Ordinamento crescente/decrescente per titolo e assegnatario.

Stato reattivo e responsabilità separate

01

INTERAZIONE

Componenti separati per logica e presentazione

02

STATO

Angular Signals aggiorna solo le parti del DOM interessate.

03

DATI

RxJS gestisce i flussi asincroni e le chiamate REST.

04

STILE

Sass centralizza temi e personalizzazioni Bootstrap 5.3

Utente → Angular Signals → gestione dello stato → aggiornamento selettivo della vista

I dati asincroni passano dai flussi RxJS alle REST API del backend.

Smart e Dumb Components: la paginazione separa le responsabilità

SMART / CONTAINER

IssueComponent

Gestisce stato e dati

- currentPage
- totalPages
- totalElements
- onPageChange()
- chiamata a IssueService

DUMB / PRESENTATIONAL

PaginationComponent

Gestisce solo la presentazione

- riceve dati con input()
- calcola le pagine con computed()
- emette pageChange
- nessuna chiamata REST
- componente riutilizzabile

Click page → pageChange → IssueComponent → IssueService → Nuova lista

Interfaccia, dominio e dati

01

CONTROLLER

DTO · validazione @NotBlank · mapping delle risposte HTTP

02

SERVICE

Regole di dominio · sicurezza · coordinamento delle transazioni

03

REPOSITORY

Spring Data JPA · query parametrizzate · accesso a PostgreSQL

I repository incapsulano l'accesso all'infrastruttura e isolano la logica applicativa.

I pattern riducono accoppiamento e rigidità

01 Dependency Injection / IoC

Spring Boot inietta le dipendenze via costruttore. Il service resta isolabile nei test.

02 Filter

SecurityFilterChain e JwtAuthenticationFilter applicano autenticazione e autorizzazione in sequenza.

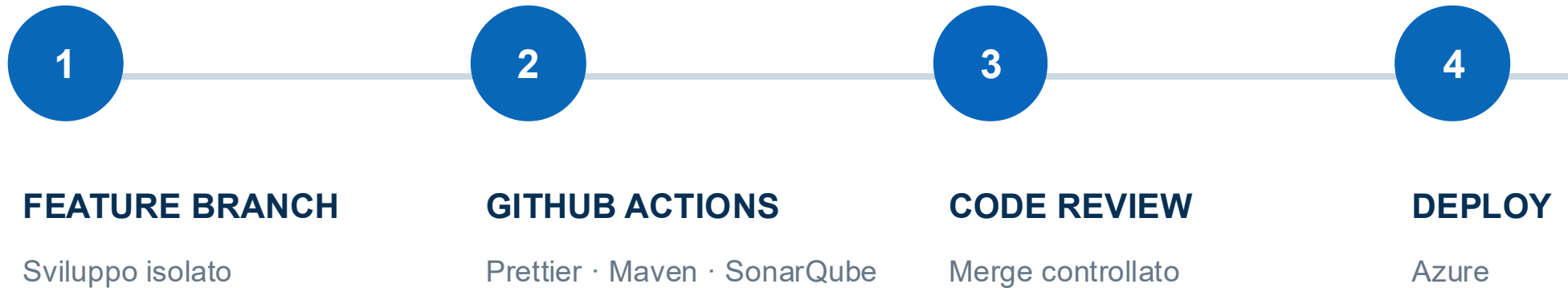
03 Strategy

IssueExporterStrategy e StreamingIssueExporterStrategy separano la logica di esportazione dal dominio; l'implementazione CSV è sostituibile o estendibile.

Risultato: componenti sostituibili, test più semplici, modifiche localizzate.

QUALITÀ E TESTING

La CI/CD del repository controlla ogni modifica prima del deploy



Automazione ripetibile

Format check, build isolata e test automatici precedono il deploy; le vecchie revisioni vengono gestite in modo controllato.

Le metriche rendono verificabile lo stato del codice

0

Security hotspots

Rating A

A

Reliability &
Maintainability

Nessun code smell critico

88.2%

Code coverage

Istruzioni coperte dai test

1.7%

Duplicazioni

Codice duplicato minimo

Copertura elevata e duplicazione contenuta sostengono l'evoluzione del progetto.

Black box e white box coprono rischi complementari

BLACK BOX

Comportamento osservabile

I test partono da input, vincoli e risultati attesi, senza considerare l'implementazione interna.

- Input validi e non validi
- Vincoli e valori limite
- Risposte HTTP e messaggi di errore

WHITE BOX

Struttura del codice

I test seguono il grafo di controllo e verificano rami, condizioni ed eccezioni.

- Rami di autorizzazione e transizione di stato
- Percorsi di successo e fallimento
- Gestione delle eccezioni

Le due prospettive si completano: requisiti all'esterno, struttura all'interno.

Le classi di equivalenza riducono casi ridondanti

DOMINIO: PROJECTKEY

CLASSE VALIDA

Progetto esistente

CLASSE NON VALIDA

Non esistente → `ResourceNotFoundException`

DOMINIO: TITLE

CLASSE VALIDA

Titolo non vuoto

CLASSE NON VALIDA

Stringa vuota → `HTTP 400 Bad Request`

DOMINIO: TAGS

CLASSE VALIDA

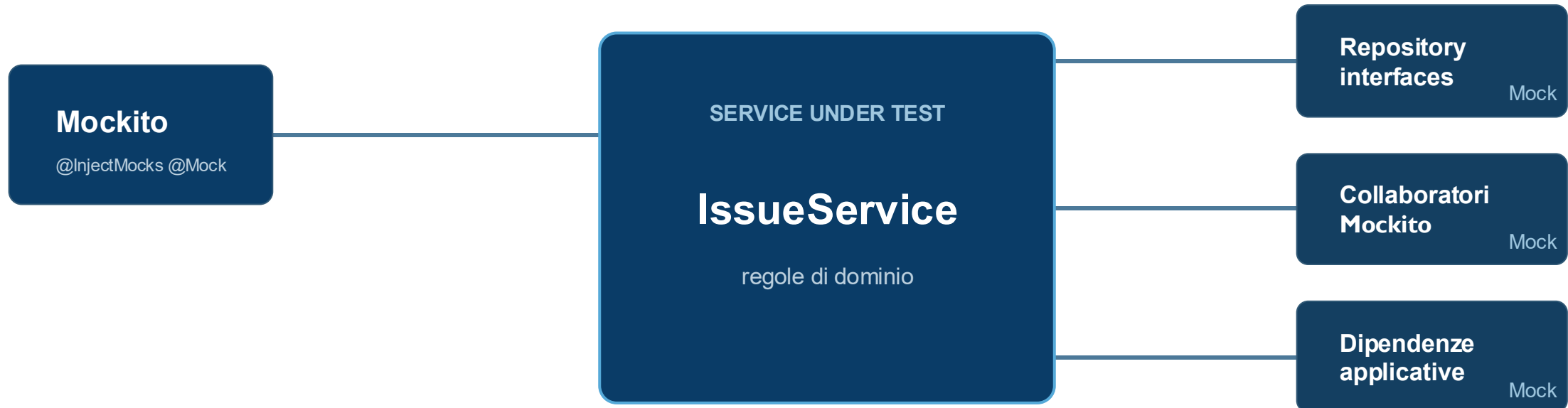
Tag dello stesso progetto

CLASSE NON VALIDA

Tag cross-project → `OperationNotAllowedException`

Un rappresentante per classe permette di coprire il dominio con un insieme contenuto di test.

Il service viene testato in isolamento



Le dipendenze esterne sono simulate: il test resta deterministico e indipendente dalla rete.

CONCLUSIONI

Una base solida e scalabile

Il progetto combina una base tecnica solida, qualità verificabile e un flusso di lavoro collaborativo.

- 01

ARCHITETTURA

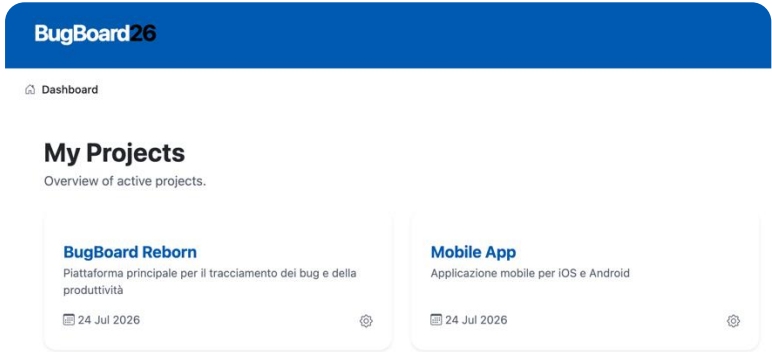
Modulare, containerizzata e fortemente tipizzata.
- 02

QUALITÀ

88.2% di coverage.
- 03

RISULTATO

Gestione collaborativa e tracciabile di issue, assegnazioni, tag e report.



Grazie per l'attenzione



Università degli Studi di Napoli
Federico II

